

Assignment 3

Team number: 8

Team members

Name	Student Nr.	Email
Alejandro Gueren Gonzalez	2727241	a.guerenagonzalez@student.vu.nl
Jakub Dryja	2732653	j.o.dryja@student.vu.nl
Daniele Roggi	2735539	d.roggi@student.vu.nl
Salvatore Pernice	2722134	s.pernice@student.vu.nl

Format: Name of each class in bold, whereas the attributes, operations, and associations as underlined text, objects are in italic.

Summary of changes from Assignment 2

Author(s): Alejandro Gueren Gonzalez, Daniele Roggi, Salvatore Pernice, Jakub Dryja

Class Diagram:

- Logical problem regarding the **MinigameScreen**. Problem is addressed in the corrected diagram by providing a corrected binary association between **MinigameScreen** and **Tamagotchi** class.

Object Diagram:

- Changing, adding and removing attributes according to the updated Class Diagram
- Highlighting the changes from Assignment 2
- Explaining the usage of the new attributes according to the logic of the newly implemented system

State Machine Diagram:

- Replacing names of attributes and functions according to the class diagram and code
- Change of the state names:
 - Sleeping → Sleep
 - Dead → Death
 - Playing → Games
 - Awake → Pet
- Play Again button on death screen is removed and a new button Exit is added
- Change of mechanisms in the Sleep state
- Change of mechanisms in the Creation state
- Minigames structure changed

Sequence Machine Diagram:

- Chngement of names of attributes and functions according to class diagram and code

and operations of almost every single class, simply due to realisations when implementing the program.

Some relationship changes within the class diagram include the association class that represents the relationship between **Tamagotchi** class and **Animal** class, where Tamagotchi class uses the AnimalFactory class in order to create an instance of a particular animal. Moreover, the **Vital** class now uses the **Observer** interface in order to notify the **Animal** whenever the instance has an empty value or when the vital changes from empty to not empty. In this case, the **Animal** implements such methods of the **observer** class, which is displayed in the class diagram by a hollow arrow. The **MinigameScreen**, now has an association with **Tamagotchi**, which was a logical error made previously. That being said, one instance of **Tamagotchi** may have any number of instances of **MinigameScreen**. This is because every **MinigameScreen** (**GuessNumberScreen** or **MemoryGameScreen**) is assigned a random value for the answer to the minigame when an instance is created. Thus every time a minigame screen is displayed, a new instance is created. We did consider having only one screen instance for the mini game screens, however this approach would be less maintainable. This is because the caller class (**Tamagotchi**) would have to randomise the answer of the minigame on every call. It is good practice to not require other classes to understand the inner workings of that given class, therefore although this approach is less efficient in terms of memory, we believe that the maintainability makes up for this design choice. The last remark regarding relationships of classes in the class diagram is the reasoning behind having 0 as a multiplicity of Tamagotchi from the screen subclasses. This must be done in our case because the Tamagotchi class is a subclass of JFrame (not mentioned in the class diagram) thus creating a new instance would result in a new frame. In order to circumnavigate this issue, we resulted in utilising public static methods so that no instance needs to be created to use the methods.

The constructor for most of the classes is shown, since this is how a lot of methods are called, therefore we would like to emphasise the point that we have used the constructor. Additionally, in some cases the principle of dependency injection is frequently used for example in certain screen classes. Although dependency injection could be avoided, this makes it more clear which class needs a certain attribute. This reduces the amount of hidden information or actions taken place in the class.

The **Tamagotchi** class has the addition of a private constructor so that no outside class is able to create an instance which provides more control. The operation createPet() has been moved from the **SelectPetScreen** to the **Tamagotchi** class. Previously, the **Tamagotchi** class had a setPet() operation that was not shown and the pet was created in the **SelectPetScreen**. This change was made to control the flow of information, in this case the pet. Now the **SelectPetScreen** has no relationship to the **Animal** class, and an association class is able to be inserted in the **Tamagotchi-Animal** relationship which is more clear.

The **SelectPetScreen** has some improvements mainly involving the animal image. A preview is shown before confirming based on the choices selected. Using images or files is notoriously tedious as it involves using filepath. In this version of the class diagram, the image is immediately stored in the currPetImg attribute of **SelectPetScreen** which is easily passed when creating the pet instead of accessing the image from the file again. One other

change which impacts the rest of the system is the introduction of the addBanner() method in the **Screen** class. Again, the **Screen** class acts as a utility class, where all subclasses inherit common and tedious operations. AddBanner() replaced the previously named addTitle() which adds the Tamagotchi banner (the signature banner of our group) onto the screen.

The **PetActionScreen** has several improvements though self explanatory. Though for clarification, the operation displayIncVital() displays the amount a specific vital has increased after performing an action.

The **Vital** class now has an added attribute of DECINTERVAL. This is essential as the value assigned determines the time interval at which every vital is decreased. The currVal is removed, though the **Vital** class utilises the JProgress bar val implicitly since **Vital** is a subclass. OFFSET is aimed to be a better named attribute for the value at which the **Vital** is supposed to increase and decrease. The two increase and two decrease operations in this class are very similar, however this is done by design in order to reduce code. ConstDecrease() constantly calls the one-time decrease() operation. This is done so that there is an operation that can be called only once to decrease the vital. This is the case where after playing a minigame, the energy vital should decrease only once. Similarly, upon incorrect selection of food for the animal, the hunger vital should increase less than the full amount which is why we have overloaded the increase() operation. It is possible to only have an increase() operation with a ratio parameter, however we decided to overload the operation because the use of ratio is very particular. By providing an operation without an argument, the operation is easy to understand and use.

As previously mentioned, each animal responds differently to the food chosen. Therefore each animal implements an abstract method of feed(), which handles the vital response depending on the food.

Lastly, there were some significant changes to the **Animal** class. Threads in our application are necessary in order to constantly decrease vitals as a background task. Which explains the threadGroup attribute. The **Animal** class now handles the situation when a **Vital** is empty or turns from empty to not empty through the notifyEmptyInc() and notifyEmptyDec() operations. The most significant change is adding a return type to all the actions the animal can do (feed(), sleep(), clean(), played()). This is necessary because in the **PetActionScreen**, the displayed value of a vital increase retrieves this value through the return type. In other words, the return value of the actions is the value by which the vital increases.

We have ensured the extensibility of adding new animals by involving hierarchy. A new animal may be added very easily without manipulating the rest of the system. This is shown in our application by involving three different animals without significant repeated code. This is also done by providing a **Vital** class where every instance represents a different vital. Since every screen (including minigame screen) is represented as a class, there is minimal impact and introducing new screens is very simple.

Application of design patterns

Author(s): Alejandro Guereña Gonzalez

	DP1
Design pattern	Singleton
Problem	Certain screens may have multiple instances within one session although this does not make logical sense (SelectPetScreen , DeathScreen). Additionally, upon returning to the same screen, PetActionScreen's instance may be reused instead of creating a new instance every time the application displays the screen.
Solution	By using the singleton design pattern on these screens, maximum one instance is created of the screen and reused when the application switches to that same screen.
Intended use	When the user creates a <u>pet</u> after confirming in the SelectPetScreen , then the PetActionScreen instance is created since there was no instance created before. From then on, once the user returns to the PetActionScreen from playing a minigame, the same instance is displayed because there can be at most one instance of PetActionScreen . Additionally, SelectPetScreen and DeathScreen are insured to only exist at most once in one session.
Constraints	By imposing the singleton design pattern, the possibility of having multiple Tamagotchi games running at the same time is not possible.
Additional remarks	The Tamagotchi class could also benefit from using the singleton design pattern in order to ensure only one instance is ever created. However since the main method is present in this class there are not a lot of moving parts thus we found it unnecessary to add certain precautions. Additionally, the Tamagotchi instance is never reused.

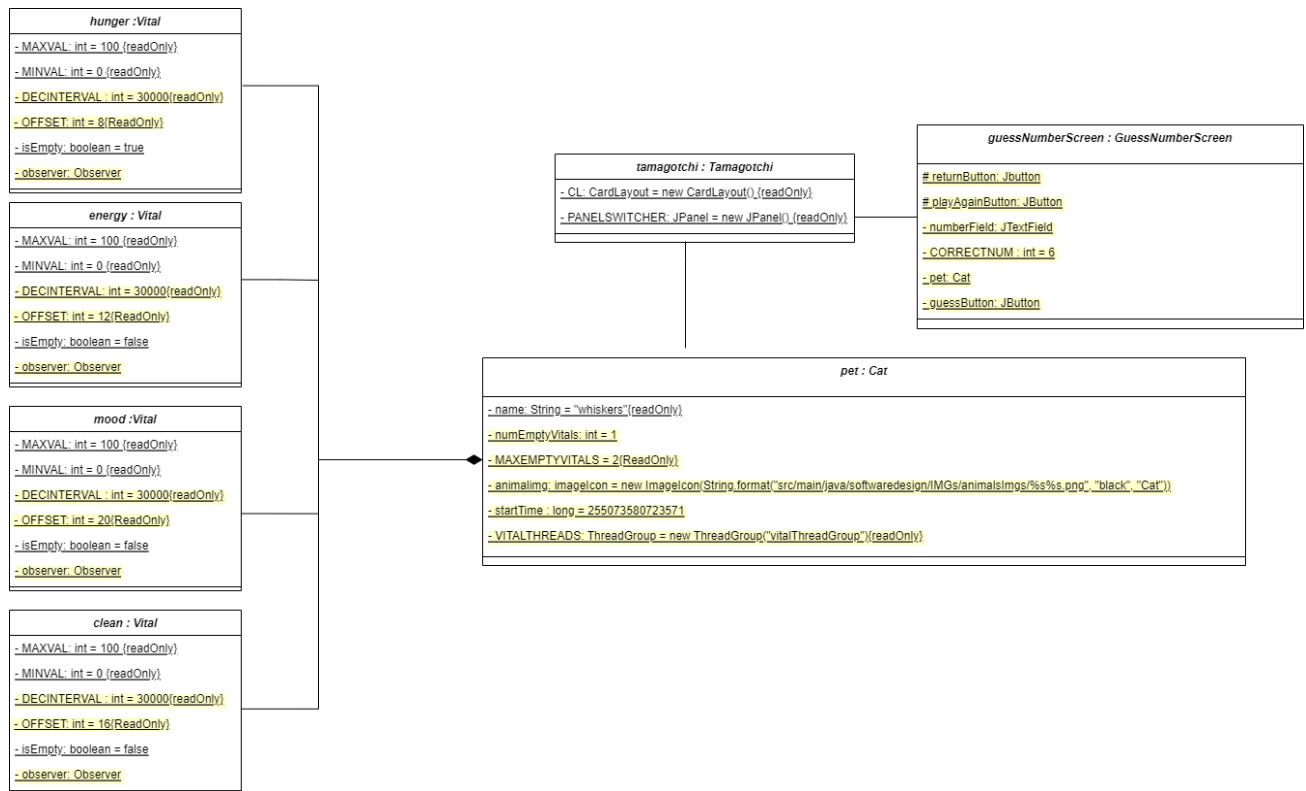
	DP2
Design pattern	Factory Method
Problem	Each subclass of Animal must declare the values by which each Vital will increase/decrease. This is inconvenient especially when wanting to compare these values to different subclasses of Animal .
Solution	By having a dedicated AnimalFactory class, the instances of Animal are easily compared and displayed. This helps with the maintainability and readability of the Animal attributes and can be changed very easily.
Intended use	The reason why the factory method is static is so that no instance of the AnimalFactory needs to be created. Therefore when an Animal instance needs to be created only the information the user chose in the SelectPetScreen is needed. Avoiding the need for passing <u>offset</u>

	variables, and then the specific Animal is returned.
Constraints	A dedicated class must be introduced into our system.
Additional remarks	One other very important solution of using the factory method is the ability to abstract the process of creating an Animal instance at design time. Creating an instance of an Animal does not require every vital <u>offset</u> value by using this design pattern. This may be done by passing the offset values through the super keyword which is why this is not mentioned in the solution.

	DP3
Design pattern	Observer
Problem	Vital class has no way to communicate to Animal class when a vital is empty or when an empty turns from empty to non empty.
Solution	With the use of an Observer class, the implicit subject class (Vital) is able to notify the Animal class when a certain state of Vital changes.
Intended use	When a vital increases, the vital class may notify Animal class that there is one more Vital that is empty. This may happen simply as the <i>pet</i> is alive since the vitals are constantly decreasing or when a minigame is played as the <i>energy</i> vital decreases. Keeping track of the number of empty Vitals is necessary for the case when the <i>pet</i> dies. This is also the case for when a Vital increases, the Vital instance may notify Animal class that a Vital went from empty to non empty. This may happen upon any action of the <i>pet</i> since it corresponds to an increase of a Vital .
Constraints	This class does not add much complexity at all. Also, since there is only one observer in our case this pattern design is far more simple than other versions. This is why there are no significant constraints that the application of the design pattern is imposing.
Additional remarks	This may also be done with a static method as done in the Tamagotchi class. However, if this is done then this method is accessible to any class without having an instance of Animal . Additionally, by using dependency injection it is clear that the Vital class may only function with an Observer class or else the class does not function as intended.

Revised object diagram

Author(s): Salvatore Pernice



The current snapshot of the system represents a state in which the user is playing the MiniGame “Guess the number”. The *PANELSWITCHER* *JPanel*(Java Swing Package) in the *Tamagotchi* *main* class has been set to display *GuessNumberScreen* via *switchScreen(String screenName)* public static method, hence all the other subclasses of abstract class “Screen” from the class diagram can be omitted in this diagram as only one panel can be displayed at a time for our program(via *CardLayout.show(JPanel, panelName)*).

The user is currently playing the Minigame and the state of the player-performance(ex: how many wrong guesses, how many attempts left) cannot be determined by the updated attributes of the class. The *pet* instance in *guessNumberScreen* is passed directly from *tamagotchi* in the constructor and it is utilised to update the mood in case of a win. The button and text fields are self explanatory as attributes and some are also general elements of *screen* and *MiniGameScreen*. We can see the randomly generated number being “6” in “CORRECTNUM” (previously named “answer” in Assignment 2).

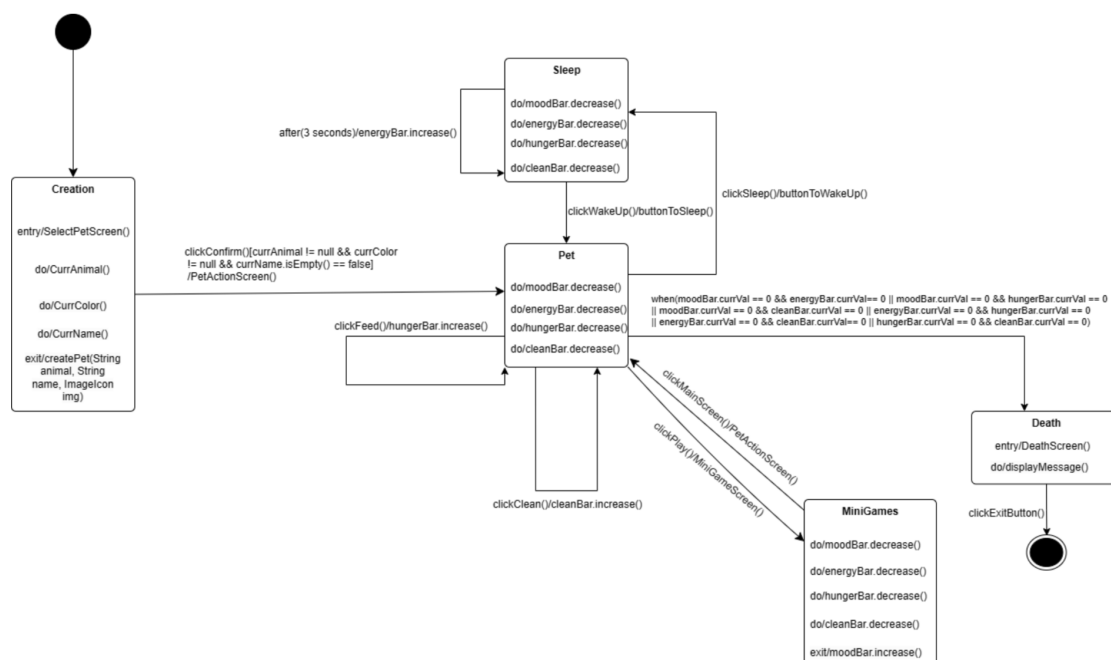
The snapshot obviously includes the living creature itself, instantiated as a *cat* by the user, and its set name “Whiskers” and image. The *ImageIcon* attribute is instantiated for the first time in the *petSelectionScreen* where the creature is firstly created abstracting the colour and appearance of the animal(in this case “black” and “Cat”) via dedicated png images. The *ImageIcon* instance is then passed to *Tamagotchi* via *Tamagotchi.createPet(...)*. *startTime* is used throughout the program to know the total lifetime in nanoseconds of the creature since creation(*currTime* - *creationTime*). In this case the time of creation was detected as being 255073580723571ns. *numEmptyVitals* and *MAXEMPTYVITALS* are used to keep track of dead vitals and to detect death of the creature. At the time of the snapshot

we can see the pet having 1 empty vital which by looking at the vitals we can see the *isEmpty* boolean being set to “true” for vital *hunger*.

The current values of the 4 vitals of the animal can no longer be seen from this interface(as previously designed in Assignment 2) as Vital is now a subclass of JProgressBar(Java Swing Package) which hold in its interface the value which can be accessed via set and get methods(to increase or decrease). The *OFFSET* attribute of the vital is what determines its decreasing/increasing rate and it is what defines the different behavioural characteristics of each animal. In our example we gave the cat vitals more emphasis on *mood* and *clean* with *OFFSET* values 16 and 20, less emphasis on hunger and energy with *OFFSET* values 8 and 12. These rates are defined per animal in the AnimalFactory class and accessed upon creation of the creature. DECINTERVAL is the same for every vital and it is the time interval between every vital Decrease. In this snapshot implementation it was set to 30000ms(vitals decrease every 30 seconds)

Revised state machine diagrams

Author(s): Daniele Roggi

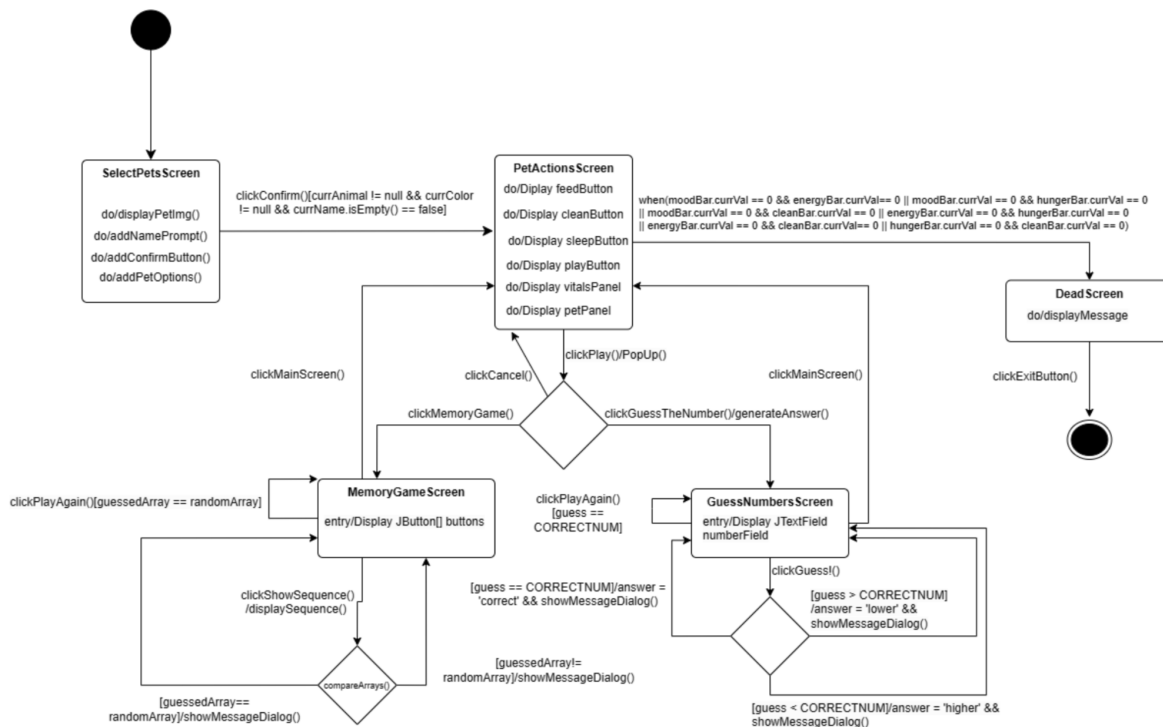


This state machine represents the class **Tamagotchi** together with various classes related to it such as **PetActionScreen**, **SelectPetScreen**, **Animal** and **DeathScreen**. This diagram is used to demonstrate the general functioning of the Tamagotchi by analysing the various states in which a pet can be, from the creation to the death. This diagram also illustrates the basic principles of the Tamagotchi by examining how actions performed by the player affect the vitals and the states of the pet.

First of all, the pet is in the **Creation** state, where the user is able to select the colour, name and type of the animal. The creation is then finalised by clicking the ‘Confirm’ button, which creates the pet and exits the state if the conditions are met. Once in the **Pet** state, the tamagotchi displays the vital bars, which continuously decrease. The player is able to interact with the system in many different ways through the buttons, the “Clean” and “Feed” buttons have the basic functionalities to increase their correspondent vital bar, the Feed functionality is practically more complex but the details have been omitted to maintain a simple and generalised diagram.

To continue with, when the user clicks on “Sleep”, it triggers a transaction to the **Sleep** state, where the vitals continue to decrease but every 3 seconds, the energy level is increased, in this state the other buttons are unusable as the pet is asleep. The pet can be easily woken up by clicking the “Wake Up” button which is now temporarily replacing the “Sleep” button, doing so, a transaction is triggered and the state switches to **Pet** again. The user can decide to perform minigames by clicking the “MiniGames” button, it is important to underline that the vitals still continue to decrease when playing minigames, once the gameplay is accomplished and the player comes back to the MainScreen which causes the state to go back to **Pet**, the mood vital bar is increased.

To conclude with, whenever two or more vitals reach the value 0, the pet enters the **Death** state, which displays an ending message and offers the possibility to the player to exit the game, an action that triggers the transaction into the final state of the diagram. Several features have been considered from Assignment 2, the main changes regards the attributes and functions name used, as a matter of fact, the latter reflect the labels in the code and the class diagram. The **Creation** state has been subject to many changes, it now indicates the variables present to the player and represents the environment where the player personalises their pet.



This state machine represents the classes related to screens such as **SelectPetScreen**, **PetActionScreen**, **DeathScreen** and the two MiniGames screens: **MemoryGameScreen** and **GuessNumbersScreen**. This diagram is used to analyse the behaviours of the screens used in our interface, relative to the actions performed by the player. The diagram also gives an overview on the functionalities of the two minigames.

To begin with, the **SelectPetsScreen** state is entered, the role of this state is to offer various functionalities to the player regarding the personalization of its pet. The confirmation of the preferences chosen triggers a transaction (if the conditions are met) to the **PetActionScreen** state, where the interface displays the vitals together with the buttons and the petPanel, in other words, an image of the personalised pet. The only button which provokes the changement of screen is the “Play” button, as a matter of fact, the player is now prompted into deciding which minigame to play and a transition to the relative state is performed. In the case in which the user clicks on “MemoryGame”, the **MemoryGameScreen** state is entered, displaying the buttons containing numbers from 1 to 5. The player is now able to initiate the minigame by clicking the showSequence button, this will display a sequence temporarily, the user will introduce the sequence he/she remembered and the latter is going to be compared with the solution. Relatively to the answer given, a message will be displayed, the message function showMessageDialog() takes as one of its input the output to the comparison of the sequences which are stored as arrays, this will help the program to display the adequate message.

In the other case, when the player clicks on “GuessTheNumber”, a transaction to **GuessNumbersScreen** state takes place, the latter displays a numberField to be fulfilled by the user. Once the player has introduced a digit, the “Guess!” button engenders the answer to be compared to the user’s guess, in all possible cases, the showMessageDialog() function is used, the latter takes as one of its input the output of the comparison between the guess and the answer (CORRECTNUM), and display a message relative to it.

The player also has the option to click on the “Cancel” button, which triggers a transaction back to the **PetActionsScreen** state.

For both minigames, the player can go back to the Main Screen and therefore enter the **PetActionsScreen** by clicking on the “Main_Screen” button. Once either of the two minigames are concluded, the player is able to play again by clicking on the “Play_Again” button.

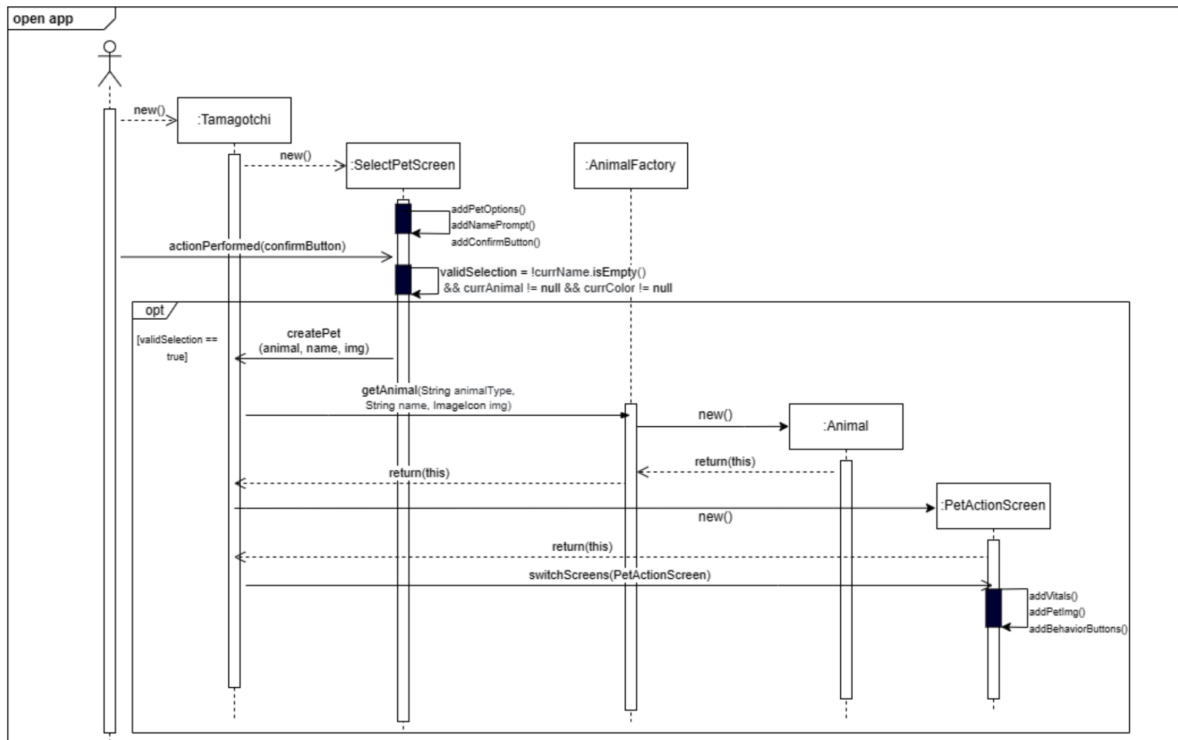
To conclude with, two or more vitals reaching the value of 0 triggers a transaction into the **DeathScreen** state, which displays an ending message.

Revised sequence diagrams

Author(s): Daniele Roggi, Jakub Dryja, Alejandro Guerena Gonzalez

Pet Creation

This sequence diagram models the process of creating a pet in the MyTamagotchi app. The goal of the diagram is to illustrate the interaction between the user and the app's system for creating and customising pets.

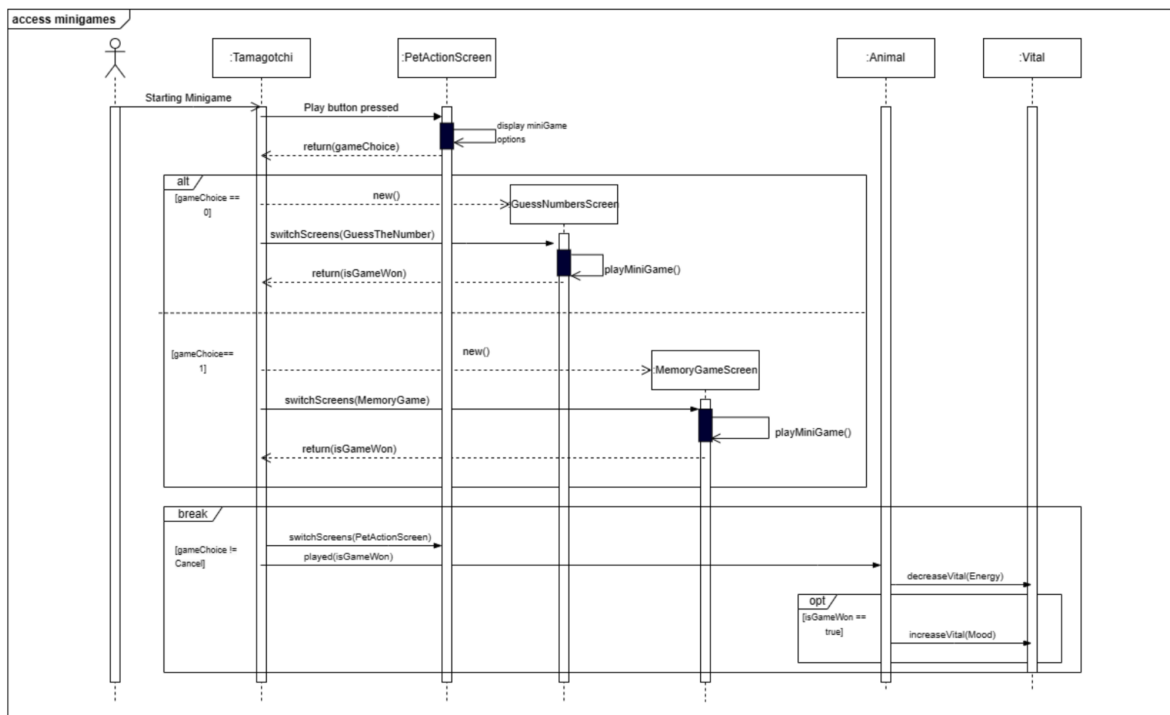


The user opens the app and creates a new instance of **Tamagotchi**, this action will immediately create an instance of **SelectPetScreen**, where the user will now be able to interact with the system and personalise its own pet. By clicking the confirmButton, the selection made by the player will be stored inside the validSection boolean. In the case the latter results are true, the pet information is transmitted from the **SelectPetsScreen** back to the **Tamagotchi**. This data will then be transferred to **AnimalFactory**, which will then create an instance of the **Animal** class. The latter instance will be returned to **Tamagotchi** and a new instance of **PetActionsScreen** will be created. The screen will finally switch to **PetActionScreen**, where Vitals, PetImg and BehaviorButtons will be added. The player is now able to interact with its personal pet and play minigames.

A large amount of changes have been made from Assignment 2, we have decided to maintain the idea that the screens should not be destroyed, we shall simply switch from one another without destruction to maintain simplicity. Moreover, the system now communicates much more, in fact, a large amount of responses have been added in order to indicate the successfulness of activities and tasks performed. Additionally, we have established that the **Tamagotchi** should not be correlated to too many classes and actions as this would reduce usability and maintainability, we tried to encapsulate as many factors as possible in order to maintain an easy to modify code and to possibly have a more extendable application for future improvements. To continue with, the usage of a new class **AnimalFactory** has been crucial for the encapsulation aspect and allowed us the application to be communicative and therefore simple.

Choosing a mini game and their impact on vitals

This sequence diagram depicts the process of selecting a mini game in the MyTamagotchi application and how it affects the vitals of the pet based on the game result. The objective of the diagram is to demonstrate the interaction with the vitals.



First of all, the player clicks the “Play” button with the intent of playing a minigame. Its choice is stored inside the gameChoice variable which is transmitted to **Tamagotchi**. An alternative interaction will now take place, in case the gameChoice is equal to 0, an instance of **GuessNumberScreen** is created, to which the screen switches to. The player plays the minigame and the result of the gameplay is stored inside the boolean isGameWon. Differently, if the gameChoice is equal to 1, an instance of **MemoryGame** is created, to which the screen switches to. The result of the minigame is once again stored inside isGameWon. To continue with, if the gameChoice is different than ‘Cancel’, the screen switches back to **PetActionScreen** and the played(result) is executed. The energy vital is going to be decreased and in case the isGameWon boolean is true, the mood vital will also be decreased.

The general design of the diagram has remained the same from assignment 2, we have come to the conclusion that creating an instance of the **GuessNumbersScreen** and an instance of the **MemoryGameScreen** every time the user plays the relative minigame is a simple solution for our application. Moreover, activation bars have been implemented to specify the activities of the classes which makes the application more methodic and straightforward. The principle of increasing the mood vital uniquely when the player wins the minigames is fundamental to give an additional importance to the user, in fact, its decision and skills will help substantially the living situation of its pet, effort will be rewarded and that is a key aspect we wanted to insert in our application.

Implementation

Author(s): Jakub Dryja

In terms of our implementation strategy for MyTamagotchi App, we relied heavily on the UML diagrams we created, especially the class diagrams. We aimed to maintain consistency between our diagrams and the implementation code. Additionally, a state machine diagram helped us establish the process leading up to a pet's death, while a sequence diagram provided a high-level overview of the app's behaviour and how we could track the pet's vitals after playing a minigame. We managed to create a boilerplate before creating our diagrams, allowing us to ensure our diagrams remained consistent with our core idea.

We didn't encounter any complex problems during implementation, but we focused on making sure the app ran smoothly, particularly when switching between screens. This was achieved by implementing a public method called switchScreen() in the main class, **Tamagotchi**, which enabled us to switch to the desired screen by adding a new JPanel to the JFrame. We also created helper functions in the **Screen** abstract class to make creating new screens easier. For example, addBanner() adds the app's name image to the top of the screen, while addButton() sets all the necessary button attributes without requiring a lot of code in other screens.

To track a pet's vitals, we used JProgressBar(class **Vital** extends JProgressBar) and created a new thread for each vital. This way, each vital is decreased after a set interval specified in the **Vital** class, DECINTERVAL.

The app provides various buttons with self-explanatory subtitles to perform different actions. We also implemented a way to track the lifetime of a pet by setting startTime = System.nanoTime() when a new instance of a pet is created. This allows us to calculate the lifetime while switching to the death screen.

In terms of mini-games, we aimed to have a low impact when adding new games. Therefore, we created a pop-up message with a list of available mini-games that appears after the user presses the play button. This method makes it easy to create new games, particularly with the methods that the **MiniGameScreen** abstract class provides, such as addMiniGameName() to display the name of the mini-game on the screen and addPlayReturnButton() to add core buttons for a mini-game screen.

The mini-games that we created are rather easy. For the GuessNumber mini-game, we set CORRECTNUM to a random integer from 0 to 10. The user has to guess the number by typing it into the JTextField, after which they can press the guess button to see if it is lower, higher, or correct. For the MemoryGame, there are five number buttons and a button to display the sequence. Until the user presses the button to display the sequence, the number buttons cannot be clicked. After the button is pressed, the sequence is displayed, and after three seconds, it disappears. The user can then press the number buttons in the sequence they remembered. A pop-up is displayed depending on whether the answer is correct or incorrect. After the mini-game is finished, the user can decide whether they want to play again or return to the main screen.

Finally, we have created a 2-minute video demonstrating the execution of our system. The video showcases the main features of our system, including the user interface, screen transitions, vital statistics, actions that can be performed on the Tamagotchi pet. The video can be accessed on YouTube using the following link:

https://www.youtube.com/watch?v=mz15qYM2IH0&ab_channel=JakubDryja

Overall, we have successfully implemented the MyTamagotchi App using Java programming language. We have focused on making our program run smoothly, ensuring that our code accurately reflects the intended functionality of our system.

The jar file is located in the out directory, then artifacts directory. Here you will find one directory named FinalVersion where the jar file for Assignment 3 is located. We attempted in many ways to get the images to work when running the jar file but could not succeed. We had tried using relative path names as well as a resource directory but neither had worked. Thus bare in mind that the images do not work when running the jar file.

Time logs

Team number	8		
Member	Activity	Week number	Hours
Salvo	Keep Track Of Implementation and ClassDiagram Changes	7	6
Salvo	Finish new Object Diagram from updated ClassDiagram	7	4
Salvo	Explain implemented system Logic according to the Object Diagram	8	4
Jakub	Making animal graphics	6	3
Jakub	Implementation explanation	8	3
Jakub	Working on the implementation	6-8	20
Jakub	Creating video demonstrating the execution of our system	8	1
Daniele	Changing State Machine diagrams according to the implementation	7	7
Daniele	Changing Sequence Machine diagrams according to the implementation	8	7
Daniele	Describing the State Machine diagrams	7	2
Daniele	Describing the Sequence Machine diagrams	8	2
Alex	Working on the implementation	6-8	20
Alex	Class diagram	8	3
Alex	Class diagram explanation	8	4
Alex	Pattern design explanation	8	2